



Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope

Abstract

Modern servers generate large volumes of log data, making it difficult for system administrators to manually identify and analyze errors. This project proposes an Automated Log Error Monitoring System, implemented in Java, that detects recurring error patterns in massive server log files using efficient string-matching algorithms. Predefined error signatures are located with a Rabin-Karp rolling-hash multi-pattern scan, KMP supports ad-hoc pattern lookups, unmatched lines are grouped by edit-distance similarity, and a suffix array with an LCP array discovers new repeating patterns.

Detected errors are categorized by frequency and severity, and advanced algorithms make the results actionable: bitmask DP selects signatures under a monitoring budget, interval DP merges time buckets into report windows, job sequencing prioritizes remediation, and network flow assigns tickets to on-call engineers. A 2-approximation Vertex Cover, randomised algorithms, and parallel primitives complete the analysis. The system also supports continuous monitoring of log files, detecting newly generated errors in real time, and generates summary reports containing error type, occurrence frequency, and severity level.

Introduction

- Modern servers and applications generate a large number of log files containing information about system activities, warnings, and errors.
- Manually analyzing large log files is difficult, time-consuming, and inefficient.
- Important error messages may be missed when logs contain thousands or millions of entries.
- Recurring errors can indicate underlying problems in servers, applications, databases, or networks.
- The proposed system automatically scans log files and detects predefined error signatures using a **Rabin-Karp** rolling-hash multi-pattern scan.
- **KMP** supports ad-hoc lookups, **edit distance** groups similar unmatched lines, and a **suffix array + LCP array** discovers new patterns.
- Detected errors are categorized based on their **type, frequency, and severity** (Low, Medium, High, Critical).
- **DP, network flow, Vertex Cover approximation, randomised and parallel algorithms** turn detections into decisions.

Literature Survey

- Large server logs are difficult to analyse manually.
- Traditional searching methods can be slow for large log files.
- **KMP** efficiently searches for individual patterns in linear time.
- **Rabin-Karp** uses a rolling hash to scan for multiple patterns.
- **Edit distance** clusters similar lines; **suffix array + LCP** finds new patterns.
- **DP, network flow, and approximation** algorithms support optimisation and assignment.
- This project combines these DSA concepts to automate log error detection.

Problem Statement and Objectives

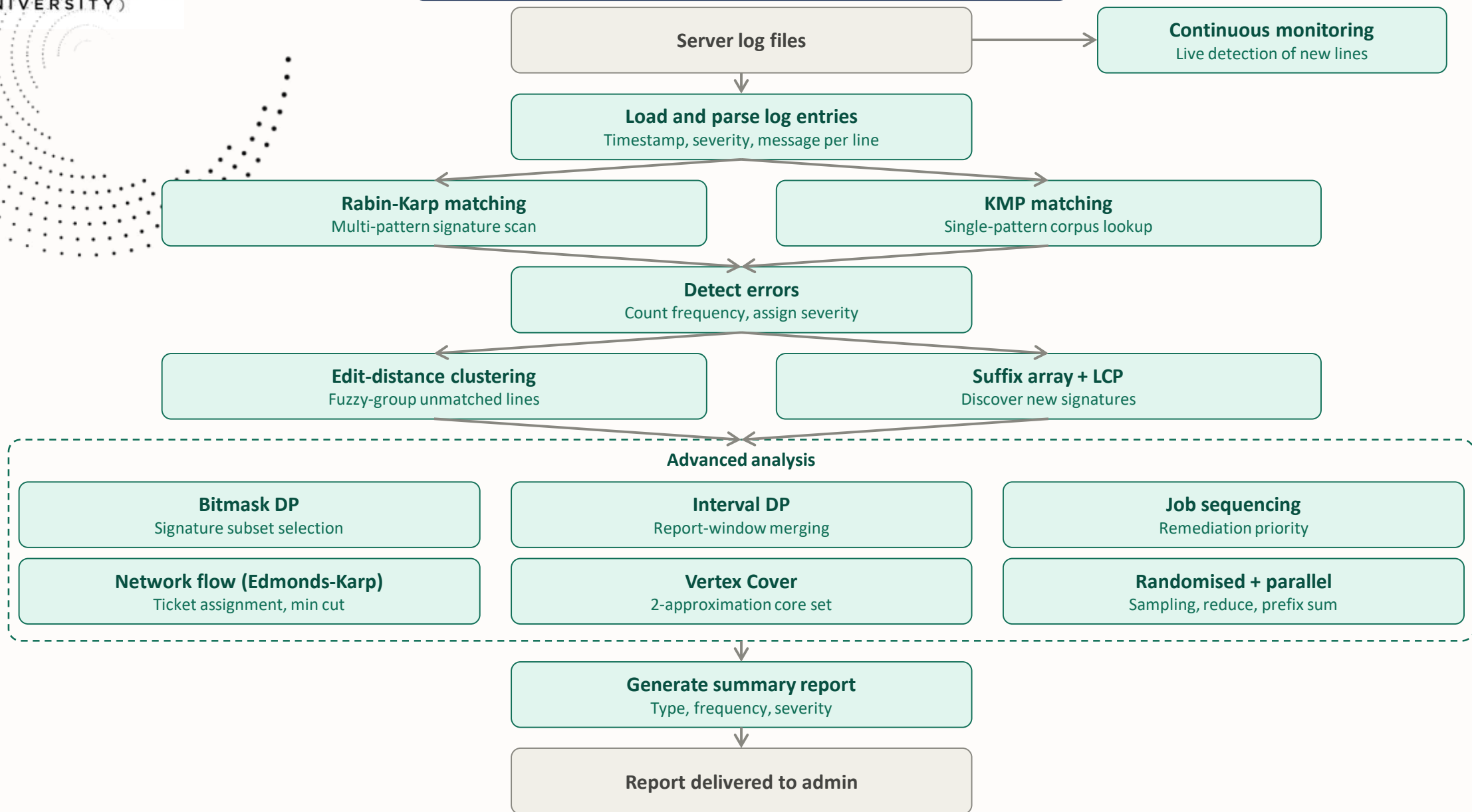
Problem Statement

- Server log files can contain a very large number of entries, making manual error detection difficult.
- Important errors may be overlooked during manual log analysis.
- Recurring errors are difficult to identify without automatically tracking their frequency.
- Administrators need a simple way to categorize errors according to their severity and occurrence.
- There is a requirement for continuous monitoring to detect newly generated errors in log files.

Objectives:

- To automatically analyse server log files.
- To detect predefined error signatures using **Rabin-Karp**, with **KMP** for single-pattern lookup.
- To cluster unmatched lines by **edit distance** and discover new patterns with **suffix array + LCP**.
- To identify recurring errors, calculate their frequency, and classify them by severity.
- To optimise monitoring and reporting using **bitmask DP**, **interval DP**, and **job sequencing**.
- To assign tickets with **network flow** and select a core monitoring set with **Vertex Cover**.
- To support continuous monitoring and generate a summary report of detected errors.

Flow Chart



Results

- The system successfully reads and processes server log files.
- Predefined signatures are detected by a **Rabin-Karp** scan; **KMP** searches the full log corpus.
- Repeated occurrences are counted, and errors are categorised by severity (Low to Critical).
- Unmatched lines are grouped by **edit distance**; **suffix array + LCP** proposes new signatures.
- **Bitmask DP** picks signatures within a budget; **interval DP** merges buckets into report windows.
- **Job sequencing** ranks remediation tasks; **network flow** assigns tickets and finds bottlenecks.
- **Vertex Cover** (2-approximation), randomised, and parallel algorithms complete the analysis.
- Continuous monitoring detects newly appended log lines in real time.
- A summary report is generated containing:
 - Error Type
 - Number of Occurrences
 - Severity Level
- The project shows how efficient DSA techniques process large text data.

Hardware and Software requirements

Hardware Requirements

- Computer or Laptop
- Minimum 4 GB RAM
- Minimum Dual-Core Processor
- Storage space for log files and generated reports

Software Requirements

- Operating System: Windows, Linux, or macOS
- Programming Language: **Java** (*JDK 14 or later*)
- IDE: VS Code / IntelliJ IDEA / Eclipse
- Sample server log file for testing (data/sample_server.log)
- Java standard libraries: java.nio.file, java.time, and Fork/Join

Conclusion

- The Automated Log Error Monitoring System provides a simple solution for analysing large server log files.
- It automates the detection of predefined error signatures using **Rabin-Karp** and **KMP** string matching.
- **Edit distance** clustering and **suffix array + LCP** discovery catch similar and previously unknown errors.
- Error frequency analysis and severity-based categorisation help prioritise critical issues.
- **DP, job sequencing**, and **network flow** turn detections into monitoring, remediation, and assignment plans.
- **Vertex Cover** approximation, randomised, and parallel algorithms add provable, sampled, and scalable analysis.
- Continuous monitoring detects new errors in real time, reducing manual effort and speeding up fault identification.
- Overall, the system demonstrates how Data Structures and Algorithms can be applied to solve real-world system monitoring problems.

Future Scope

- Integration with live server and application log streams beyond file watching.
- Support for a larger number of log formats.
- Automatic promotion of patterns discovered by suffix array + LCP into new known signatures.
- Integration of Machine Learning for anomaly and error detection.
- Development of a graphical dashboard for visualising error statistics.
- Real-time alert notifications through email or messaging services.
- Support for distributed and cloud-based server monitoring.
- Loading engineer capacities and error categories from configuration instead of fixed values.
- Integration with databases to store historical error information.
- Analysis of error trends to predict possible future system failures.

Thank you